Sprint 11 · Reto «Visión artificial para el cliente»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. TurisData recibe **cientos de fotos** de alojamientos cada semana y una persona las clasifica a mano (hotel, apartamentos, casa rural, camping, piscina). Quiere saber si un **modelo de visión artificial** (una IA que trabaja con imágenes) puede **pre-clasificarlas** y, sobre todo, **en qué se equivoca**, para decidir cuánto puede fiarse de él y **qué fotos debe revisar una persona**. Lo importante no es el acierto, sino **saber cuándo no fiarse**: un modelo que acierta el 90 % pero no sabe cuándo falla es peligroso; uno que acierta el 85 % y dice «esto no lo tengo claro, que lo revise una persona» es útil. Os pide tres cosas: un **modelo** entrenado con *transfer learning* (reutilizar una red ya entrenada y adaptarla a vuestras clases), un **demostrador** (una función que recibe una foto y responde clase, confianza o «revisar a mano») y un **análisis de errores** de 1–2 páginas.

Cómo se trabaja. No hace falta experiencia previa: el cuaderno `S11_04` (que practicáis el día 2) trae casi todo el código y cinco ejercicios cortos; vuestro trabajo es completarlo, entender los resultados y **mirar las fotos que el modelo falla**. Por defecto se usa un conjunto de **dibujos sintéticos** (no son fotos reales) y, si queréis, uno propio con licencia. En clase tenéis 10 minutos de planning y **25 minutos de reto el día 1**, **80 minutos el día 2** y 30 de entrega, *review* y retro; lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (Python con TensorFlow/Keras y scikit-learn ya instalados)	Ejecutar el cuaderno sin instalar nada	En el navegador, con una cuenta de Google. Colab puede desconectarse: ejecutad otra vez desde arriba
El cuaderno del reto `S11_04_reto_vision_cliente`	Es el esqueleto: genera el conjunto de fotos, parte los datos, entrena, evalúa y trae el demostrador	Colab. Haced una copia (*Archivo → Guardar una copia en Drive*); su código explicado
Cuadernos de las prácticas: `S11_01` (CNN) y `S11_02` (*transfer learning*)	Las pistas para congelar la red, entrenar la «cabeza» y hacer el ajuste fino	Colab
Presentación y apuntes `S11.2` · apuntes de visión por computador	Repasar qué es una CNN y qué es el *transfer learning*	En la carpeta del sprint
Las fotos sintéticas	5 clases (hotel, apartamentos, casa rural, camping, piscina), 60 dibujos por clase; algunos con la etiqueta dudosa a propósito	Las **genera el propio cuaderno** la primera vez (o el script `generar_dataset_sintetico.py`)
Fotos propias (opcional)	Probar con fotos reales: hechas por vosotros o con licencia abierta, **sin personas reconocibles**	Una carpeta por clase; apuntad autor y licencia de cada una
Cómo entregar en GitHub	Subir los ficheros a `sprint-11/` y subir el cuaderno de Colab	Guía de entrega

Recurso	Para qué lo usaréis	Dónde está
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender errores y pedir ideas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el cuaderno) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-11/` : 3 ficheros del equipo (`reto_vision_cliente.ipynb` , `analisis_errores.md` , `README.md`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **3 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 6). «El reto» no es un fichero aparte: son esos tres ficheros, y el **cuaderno contiene el modelo y el demostrador**. Las fotos (sintéticas o propias) **no se suben**: el cuaderno las vuelve a generar y, si usáis fotos propias, el README cuenta de dónde salen y con qué licencia. El modelo guardado (`.keras`) solo si lo guardáis. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-11/`** .

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`reto_vision_cliente.ipynb`	**Lo modificáis.** Parte del cuaderno `S11_04` (esqueleto con los ejercicios 1 a 5 y el demostrador)	Completar los ejercicios, entrenar y evaluar con vuestros datos, y ejecutarlo entero de arriba abajo. Al bajarlo de Colab, que conserve el nombre `reto_vision_cliente.ipynb`	**Sí** · `sprint-11/`
`analisis_errores.md`	**Lo creáis.** No hay plantilla: seguid los seis apartados del paso 5	Escribir el informe de 1–2 páginas (métricas por clase, matriz de confusión, al menos 6 errores, recomendación). Si preferís PDF, se sube en lugar del `.md`	**Sí** · `sprint-11/`
`README.md`	**Lo creáis.** No hay plantilla: seguid las secciones del paso 5	Cómo se ejecuta, de dónde salen las fotos, uso de la IA, quién hizo qué y la revisión con la rúbrica	**Sí** · `sprint-11/`
`clasificador_alojamientos.keras`	**Lo genera el cuaderno** con `modelo.save("clasificador_alojamientos.keras")`	Nada si no lo queréis: el cuaderno ya reproduce el modelo	Solo si lo guardáis · `sprint-11/`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-11/`
Carpeta `datos_sinteticos/alojamientos/` (una subcarpeta por clase)	**Lo genera el cuaderno** (o `generar_dataset_sintetico.py`) la primera vez que lo ejecutáis	Nada: se crea sola y se puede borrar y volver a crear	**No se sube**
`generar_dataset_sintetico.py`	**Ya hecho.** Script del curso que crea las mismas fotos sintéticas	Nada: solo si queréis generarlas fuera de Colab	**No se sube**
Fotos propias (si las usáis)	**Las creáis vosotros** (fotos hechas por el equipo) o las bajáis con licencia abierta	Una carpeta por clase; apuntar autor y licencia; sin personas reconocibles	**No se suben**: autor y licencia van en el README

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Conjunto de fotos	Una carpeta por clase, mínimo 30 fotos por clase (mejor 100 o más) y sin personas reconocibles. Se comprueba con `contar_por_clase` (ejercicio 1). El sintético del cuaderno vale	Mínimo	`reto_vision_cliente.ipynb` · sección 1
Cuaderno ejecutable	Se ejecuta de arriba abajo (*Entorno de ejecución → Reiniciar y ejecutar todo*) con la semilla fija (`SEMILLA = 42`)	Mínimo	`reto_vision_cliente.ipynb`
Modelo con *transfer learning*	Red preentrenada **congelada** más una cabeza nueva con una neurona por clase (`modelo_transfer`, ejercicio 2)	Mínimo	`reto_vision_cliente.ipynb` · sección 2
Métricas por clase y matriz de confusión	Precisión y *recall* de cada clase (ejercicio 3), la matriz de confusión de la prueba y una frase que diga qué clases se confunden	Mínimo	`reto_vision_cliente.ipynb` · sección 3 y `analisis_errores.md`
Tres errores analizados	Tres fotos mal clasificadas, cada una con una hipótesis de por qué	Mínimo	`analisis_errores.md`
Partición y prueba	Entrenamiento, validación y prueba separados con la misma semilla; **la prueba se usa una sola vez**	Estándar	`reto_vision_cliente.ipynb`
Referencia «desde cero»	La misma red sin preentrenar, entrenada con los mismos datos, para ver cuánto aporta el *transfer learning*	Estándar	`reto_vision_cliente.ipynb` · `analisis_errores.md`
Seis errores analizados	Al menos 6 fotos mal clasificadas, cada una con la clase verdadera, la predicha, una hipótesis **que se vea en la foto** y una mejora	Estándar	`analisis_errores.md`
Demostrador	`clasificar_foto(ruta)` devuelve clase, confianza y «revisar a mano» si la confianza es menor que el umbral	Estándar	`reto_vision_cliente.ipynb` · sección 6
Datos y riesgos de IA responsable	Origen y licencia de las fotos y al menos dos riesgos concretos (fotos poco representativas, clases poco frecuentes, etiquetas dudosas, privacidad, licencias) con una medida cada uno	Estándar	`README.md` y `analisis_errores.md`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Informe	1–2 páginas con los seis apartados del paso 5. En el mínimo, versión corta (matriz y tres errores)	Mínimo	`analisis_errores.md`
README	Cómo se ejecuta, de dónde salen las fotos, uso de la IA (qué pedisteis y qué revisasteis) y quién hizo qué. En el mínimo, versión corta	Mínimo	`README.md`
Git	Al menos 3 *commits* con mensaje claro, de personas distintas, sin claves ni datos personales	Mínimo	Historial del repositorio del equipo
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa cuaderno, informe y README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-11/`
Umbral elegido con datos	Probar varios umbrales y medir la **cobertura** (qué parte de las fotos decide el modelo solo) y el acierto de esas fotos; justificar el umbral elegido	Extra	`analisis_errores.md` · `reto_vision_cliente.ipynb`
Fotos reales con MobileNetV2	La rama de Colab (sección 7 del cuaderno) con fotos reales con licencia, comparada con el modelo anterior	Extra	`reto_vision_cliente.ipynb` · `README.md`
Mejora probada	Una mejora propuesta en el informe (más fotos de una clase, aumento de datos, revisar etiquetas…) que se prueba y se mide	Extra	`analisis_errores.md`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay cuaderno, pero no se ejecuta de arriba abajo o no hay un modelo con resultados	El cuaderno se ejecuta; las clases se han contado; hay un modelo con base congelada y cabeza nueva, y métricas por clase y matriz de confusión de la prueba	Además, partición fija con semilla y **prueba usada una sola vez**; la referencia «desde cero» se compara con los mismos datos; `clasificar_foto` devuelve clase, confianza y «revisar a mano»	Además, el umbral se elige midiendo cobertura y acierto en validación, o hay una segunda base (MobileNetV2) comparada con la primera
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en la sesión de Colab de quien lo hizo; no se sabe de dónde salen las fotos	El README dice cómo abrir el cuaderno, pero falta algo: el origen de las fotos, la semilla o el orden de ejecución	Otra persona, siguiendo solo el README y *Reiniciar y ejecutar todo*, obtiene las **mismas cifras**; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, el README indica las versiones (Python y TensorFlow), cómo conseguir las mismas carpetas de fotos y qué esperar al terminar
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	Solo hay una cifra de acierto; no se dice en qué se equivoca	Las métricas por clase y la matriz dicen qué clases se confunden; hay 3 errores con una hipótesis cada uno	Al menos **6 errores** con foto, hipótesis que se ve en la imagen y mejora; la recomendación dice qué fotos se automatizan y cuáles se revisan a mano, con el umbral, sin jerga	Además, el umbral está justificado con números (cuántas fotos revisa una persona frente a cuántos fallos se evitan) y la mejora propuesta se ha probado
IA responsable (¿se sabe qué hizo la IA y qué riesgos hay?)	No dice si se ha usado IA ni de dónde vienen las fotos	Dice si se ha usado IA y de dónde vienen las fotos, sin riesgos concretos	Dice **qué se pidió a la IA, qué se revisó y qué se cambió**; da origen y licencia de las fotos y **dos riesgos concretos** con su medida	Además, cuenta un fallo concreto (por ejemplo, un error con confianza alta) y qué se hace con él, y avisa de que sintético no es lo mismo que fotos reales

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** el cuaderno con *Entorno de ejecución → Reiniciar y ejecutar todo*, siguiendo solo lo que dice el README. Anotar si algo falla o si las cifras no coinciden con las del informe.
2. **Leer el informe y el README como si fuerais el cliente**: ¿se entiende en qué se equivoca el modelo y qué fotos debe revisar una persona? Subrayar lo que no se entienda.
3. **Mirar dos errores del informe** con la foto delante: ¿la hipótesis se ve de verdad en la imagen? Que quien la escribió la explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · el cuaderno se ejecuta y la referencia sale en la misma tabla, pero la prueba se usa dos veces»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son cinco pasos, de **105 minutos en total**: el primero el día 1 (25 minutos) y los otros cuatro el día 2 (80 minutos). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa. Una idea para todo el reto: **primero se mira, después se mide**. Mirad siempre las fotos, no solo los números.

Paso 1 · Preparar los datos (día 1, 25 minutos)

1. Abrir el cuaderno `S11_04` en Colab, hacer una copia y compartirla con el equipo.
2. Decidir si usáis las fotos sintéticas del cuaderno o un conjunto propio.
3. Comprobar cuántas fotos hay por clase y mirarlas.
4. Anotar las etiquetas dudosas y ver cómo se parten los datos.

Cómo se hace

- **Copiar el cuaderno.** Abrid `S11_04_reto_vision_cliente`, *Archivo → Guardar una copia en Drive*, y compartidla con el resto del equipo para que todos puedan editar. Ejecutad **primero la celda de «Preparación»** y después las de los imports y la semilla (`SEMILLA = 42`).
- **Elegir las fotos.** Por defecto (`USAR_DATASET_PROPIO = False`) el cuaderno crea 60 dibujos por clase en `datos_sinteticos/alojamientos`. Si usáis fotos propias, poned `USAR_DATASET_PROPIO = True` y la ruta de vuestra carpeta en `RUTA_PROPIA`: **una carpeta por clase**, nombres sin espacios ni tildes (`casa_rural`), mínimo 30 fotos por clase (mejor 100 o más), hechas por vosotros o con licencia abierta, **sin personas reconocibles**. Apuntad autor y licencia.
- **Contar por clase (ejercicio 1).** Completad `contar_por_clase(ruta)`: devuelve un diccionario `{clase: nº de imágenes}`. Sirve para detectar clases vacías o desequilibradas.
- **Mirar las fotos.** Abrid unas cuantas de cada clase. ¿Hay alguna con la etiqueta dudosa (una piscina en una carpeta de hotel)? Anotadlas: el cuaderno incluye algunas **a propósito**, como pasa con las etiquetas dudosas de los datos reales.
- **Ver la partición.** La celda con `image_dataset_from_directory` reparte las fotos en **entrenamiento (60 %), validación (20 %) y prueba (20 %)** con la misma semilla. La prueba se usa **una sola vez**, al final.

Qué deberíais ver. El mensaje `Dataset sintético generado en datos_sinteticos/alojamientos` (solo la primera vez), un diccionario con las cinco clases y 60 fotos en cada una, y una línea como `entrenamiento (180, 48, 48, 3) · validación (60, 48, 48, 3) · prueba (60, 48, 48, 3)`, más una fila de ocho fotos de ejemplo con su clase.

Hecho cuando: `contar_por_clase` pasa sus comprobaciones, habéis mirado las fotos y anotado las dudosas, y todo el equipo sabe que la prueba no se toca hasta el paso 3.

Paso 2 · Entrenar con *transfer learning* y una referencia (día 2, 25 minutos)

1. Completar `modelo_transfer` (ejercicio 2): copia congelada de la base preentrenada más una cabeza nueva.
2. Entrenar el modelo y hacer el ajuste fino del último bloque.
3. Entrenar una referencia «desde cero» con la misma arquitectura.
4. Anotar la precisión de validación de las tres versiones.

Cómo se hace

- **La idea.** Una red que ya ha aprendido a ver formas y texturas se **reutiliza**: se **congelan** sus capas (no cambian) y solo se entrena la **cabeza** (las capas nuevas que dan una respuesta por clase). Después, el **ajuste fino** (*fine-tuning*) descongela el último bloque y lo ajusta con una tasa de aprendizaje muy baja. Pista: `S11 02`.
- **Ejercicio 2.** Escribid `modelo_transfer(base_preentrenada, n_clases)`: haced una copia de la base con `keras.models.clone_model`, pasadle los pesos con `set_weights(...get_weights())`, ponedle `trainable = False` y añadid la cabeza con `con_cabeza`. **La base original no debe quedar congelada**: el `assert` de la celda os lo dirá.
- **Entrenar.** Ejecutad las celdas de la sección 2: se crea la base preentrenada con una tarea auxiliar (unos 15 segundos), se entrena con la base congelada y luego con el último bloque descongelado.
- **La referencia.** La celda siguiente entrena `desde_cero`, la misma red sin preentrenar. Anotad su precisión de validación con `desde_cero.evaluate(X_val, y_val, verbose=0)[1]`.
- **Solo validación.** Aquí se mide **únicamente en validación**. La prueba no se toca todavía.
- **Extra .** Con fotos reales en Colab podéis usar la rama de MobileNetV2 (sección 7: `USAR_INTERNET = True`, `TAM` de 96 a 160 y `construir_mobilenet(N_CLASES, pesos="imagenet")`). Es opcional y necesita internet.

Qué deberíais ver. Tres cifras de precisión de validación: base congelada, tras el ajuste fino y desde cero. Lo esperable es que el *transfer learning* sea igual o mejor que la referencia y aprenda con menos fotos; si no lo es, no es un error: anotadlo y pensad por qué.

Hecho cuando: `modelo_transfer` pasa sus comprobaciones y tenéis anotadas las tres cifras de validación.

Paso 3 · Evaluación y análisis de errores (día 2, 20 minutos)

1. Evaluar en la prueba (una sola vez) y sacar las métricas por clase.
2. Mirar la matriz de confusión y completar los ejercicios 3 y 4.
3. Elegir las fotos mal clasificadas (3 como mínimo, 6 como estándar) y rellenar la tabla de errores.
4. Buscar patrones entre los errores.

Cómo se hace

- **Evaluar.** Ejecutad las celdas de la sección 3: precisión de los dos modelos, `classification_report` y la **matriz de confusión** (filas = clase real, columnas = clase predicha; lo que queda fuera de la diagonal son los errores).- **Ejercicio 3.** Con la matriz `cm`, calculad `recall_clase` (la diagonal entre la suma de cada fila), `precision_clase` (entre la suma de cada columna) y `clase_mas_dificil`. Con ello sabréis qué clase falla más.
- **Ejercicio 4.** Completad `indices_errores(y_real, prob)`: devuelve los índices de las fotos mal clasificadas, **de más a menos confianza**. Los primeros son los más preocupantes: el modelo estaba seguro y se equivocó. La celda siguiente os enseña hasta diez, con la clase real, la predicha y la confianza.
- **Rellenar la tabla de errores.** Elegid las fotos y copiad la tabla en `analisis_errores.md`. En «Foto» poned la carpeta de la clase real y el nombre del fichero, por ejemplo `camping/img_031.png`:

Foto	Verdadera	Predicha	Confianza	Hipótesis de causa	Mejora propuesta
`piscina/img_012.png` (ejemplo inventado)	Piscina	Hotel	81 %	El edificio ocupa media imagen y la piscina es muy pequeña	Más ejemplos con la piscina en primer plano

- **Buscar patrones.** ¿Los errores se juntan en un par de clases? ¿Son fotos oscuras o con ruido? ¿Hay etiquetas dudosas? ¿Fondos parecidos? La celda de los pares (`value_counts`) os dice cuáles se confunden más. Una hipótesis vale cuando **se ve en la foto**: «por falta de datos» no basta.

Qué deberíais ver. La precisión en la prueba de los dos modelos, el informe por clase, la matriz de confusión, entre cero y diez fotos mal clasificadas con su clase real y predicha, y la lista de pares más confundidos.

Hecho cuando: tenéis las métricas por clase y la matriz interpretada en una frase, y la tabla con al menos 3 errores (6 si vais al estándar), cada uno con hipótesis y mejora. **A partir de aquí la prueba no se vuelve a usar para decidir nada.**

Paso 4 · Demostrador y umbral de confianza (día 2, 15 minutos)

1. Probar `clasificar_foto` con 2 o 3 fotos de la prueba.
2. Medir cobertura y acierto para varios umbrales con el ejercicio 5.
3. Elegir el umbral y apuntar la recomendación.

Cómo se hace

- **El demostrador.** Está en la sección 6 del cuaderno (queda bajo la línea «en casa» de la práctica, pero en el reto sí se hace). `clasificar_foto(ruta_imagen, umbral=0.7)` carga una foto y devuelve la **clase**, la **confianza** (lo seguro que está el modelo, de 0 a 1) y las probabilidades; si la confianza es menor que el **umbral**, en vez de la clase dice «revisar a mano». Probadla con 2 o 3 rutas de fotos de la prueba (`rutas_pru[0]`, `rutas_pru[1]`…) y ved qué responde.
- **Ejercicio 5.** Completad `cobertura_y_precision(prob, y_real, umbral)`: devuelve la **cobertura** (qué parte de las fotos decide el modelo solo) y la **precisión de las aceptadas** (qué parte de esas acierta). Un umbral alto da menos fotos automáticas, pero más fiables.
- **Elegir el umbral.** Para el estándar basta con el umbral que trae el cuaderno y explicar qué hace. Para el extra , calculad la tabla con varios umbrales **sobre validación** (`modelo.predict(X_val, verbose=0)`), porque la tabla del cuaderno usa las probabilidades de la prueba. Elegid el que equilibre fotos revisadas y errores evitados.
- **Una frase para el cliente.** Escribidla ya, con números: «Con un umbral de …, el modelo decide solo el … % de las fotos y acierta el … % de ellas; el resto lo revisa una persona».

Qué deberíais ver. La respuesta de `clasificar_foto` para cada foto, con su confianza (a veces «revisar a mano»), y una tabla con la cobertura y la precisión de las aceptadas para cada umbral probado.

Hecho cuando: `clasificar_foto` funciona con 2 o 3 fotos delante del equipo, `cobertura_y_precision` pasa sus comprobaciones y tenéis anotado el umbral elegido con su frase.

Paso 5 · Informe, README y entrega (día 2, 20 minutos)

1. Completar `analisis_errores.md` con lo que ya habéis ido apuntando.
2. Crear el `README.md`.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado en el README.
4. Reiniciar y ejecutar todo el cuaderno, bajarlo y subir los ficheros a `sprint-11/`.

Cómo se hace

- **El informe** (`analisis_errores.md`, 1–2 páginas), con estos seis apartados: (1) **qué se ha hecho y con qué datos** (origen y licencia); (2) **resultados**: tabla de métricas por clase y comparación con la referencia; (3) **la matriz de confusión interpretada**; (4) **los errores** (la tabla del paso 3); (5) **la recomendación al cliente**: qué se automatiza, qué fotos se revisan a mano y con qué umbral; (6) **al menos dos riesgos de IA responsable** con su medida. Para el nivel mínimo bastan los apartados 1, 3 y 4.
- **El README.** No hay plantilla. Para el nivel mínimo, estas secciones en pocas líneas:

Sección del README	Qué escribís
Qué es	Dos o tres frases: el cliente, qué clasifica el modelo y qué se entrega
Cómo se ejecuta	Abrir `reto_vision_cliente.ipynb` en Colab, *Entorno de ejecución → Reiniciar y ejecutar todo*, semilla 42. Para el , las versiones (`print(tf.__version__)`)
Datos	Sintéticos (los genera el cuaderno) o propios, con **autor y licencia** de las fotos; cuántas fotos por clase
Uso de IA y límites	Qué pedisteis a la IA, qué revisasteis y qué cambiasteis; los dibujos sintéticos **no son fotos reales**
Quién hizo qué	Una línea por persona

- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y añade al final del README esta sección: un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

```
## Revisión con la rúbrica
**Revisado por:** <nombre> · **Fecha:** <dd/mm/aaaa>

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | <1–4> | <p. ej. «se ejecuta de arriba abajo; la prueba se usa dos veces»> |
| Reproducibilidad | <1–4> | <¿salen las mismas cifras siguiendo solo este README?> |
| Análisis y comunicación al cliente | <1–4> | <¿se entiende en qué se equivoca y qué se revisa a mano?> |
| IA responsable | <1–4> | <¿dice qué se pidió, qué se revisó y de dónde salen las fotos?> |

**Lo primero que mejoraríamos:** <una cosa concreta>
```

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) en Colab, *Entorno de ejecución* → *Reiniciar y ejecutar todo* y luego *Archivo* → *Descargar* → *Descargar .ipynb*; (2) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-11`**; (3) *Add file* → *Upload files*, arrastrad el fichero y pulsad **Commit changes**. El `README.md` y el informe se pueden escribir con *Add file* → *Create new file*. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `reto_vision_cliente.ipynb` («Añado cuaderno con el modelo y el demostrador»), la B sube `analisis_errores.md` («Añado análisis de errores») y la C sube `README.md` («Añado README»). Los nombres de archivo, **en minúsculas y sin espacios**.

Qué deberíais ver. En la carpeta `sprint-11` del repositorio, los tres ficheros del equipo y una hoja de autoevaluación por persona (**3 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos.

Hecho cuando: otra persona podría ejecutar el cuaderno siguiendo solo el README, la sección «Revisión con la rúbrica» está rellena y los tres *commits* están en el repositorio.

Después, 30 minutos de entrega y review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-11/`, y el equipo enseña, en unos 5 minutos, el demostrador funcionando con 2 o 3 fotos, la matriz de confusión y tres errores comentados («el modelo confunde X con Y porque… y propondríamos…»). Después, la retro.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **suena muy segura aunque se equivoque** y no ha visto vuestras fotos. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea o por un error del informe, tenéis que saber explicarlo.
- **Las hipótesis se comprueban mirando la foto.** La IA propone causas típicas; vosotros dejáis solo las que **se ven en vuestras fotos**.
- **Declaradlo** en el README: qué pedisteis, qué revisasteis y qué cambiasteis.
- **Nada de datos personales ni claves** en una IA, y **no subáis fotos con personas** a ningún servicio.

Para qué	Ejemplo de petición	Qué comprobáis
Entender el *transfer learning*	«Explícame transfer learning con la analogía de un cocinero experto que aprende un plato nuevo»	Que podéis contarlo con vuestras palabras
Interpretar una matriz	«Mi modelo confunde mucho camping con casa rural. ¿Qué causas habituales puede haber?»	Que las contrastáis mirando las **fotos reales**
Entender un error de Keras	«Me sale este error al cargar las imágenes: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que entendéis la causa y lo arregláis vosotros
Revisar el informe	«Léelo como si fueras el cliente y dime qué no se entiende»	Que no cambia ninguna cifra

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`AssertionError` al ejecutar un ejercicio	La función no devuelve lo que pide el enunciado	Leer el mensaje del `assert`: dice qué esperaba. Probar con un ejemplo pequeño
La base original queda congelada	Se pone `trainable = False` sobre la base y no sobre la copia	Trabajar con la copia (`clone_model`); el ejercicio 2 lo comprueba

Error	Por qué ocurre	Cómo evitarlo
Fotos casi idénticas en entrenamiento y prueba	Se parte sin cuidado o se generan variantes antes de partir	No tocar la partición del cuaderno; si hacéis aumento de datos, solo con las fotos de entrenamiento
Pocas fotos en alguna clase	Es difícil conseguirlas	Mínimo 30 por clase; si no llegáis, avisad del límite en el informe
Fiarse solo de la exactitud global	Es un solo número y esconde las clases que fallan	Mirar las **métricas por clase** y la matriz de confusión
Analizar errores sin mirar las fotos	Es más cómodo	Abrir las fotos mal clasificadas una a una
Usar la prueba para decidir	Se prueba «a ver qué sale» varias veces	Decidir con validación; la prueba se mira **una vez**
Umbral de confianza «a ojo»	Se pone 0,5 sin más	Medir cobertura y acierto de varios umbrales
Usar fotos de internet sin licencia	«Total, es un curso»	Solo fotos propias o con licencia abierta; guardar autor y licencia

Ayuda 3 · Si os atascáis

1. **Colab dice que no hay memoria o se desconecta:** reducid el tamaño de las imágenes o el número de fotos, y usad *Entorno de ejecución → Reiniciar y ejecutar todo*.
2. **El modelo acierta siempre la misma clase:** hay desequilibrio; comprobad con `contar_por_clase` cuántas fotos hay en cada clase.
3. **No sabemos explicar un error:** mirad la foto como si fuerais el modelo: ¿qué rasgo la haría parecerse a la otra clase?
4. **Hay menos de 6 errores en la prueba:** analizadlos todos y completad hasta 6 con errores de validación, diciéndolo en la tabla. Preguntad al docente si dudáis.
5. Repasad el apartado del paso en el que estáis y el código explicado del cuaderno; después, preguntad al equipo y, por último, al docente.